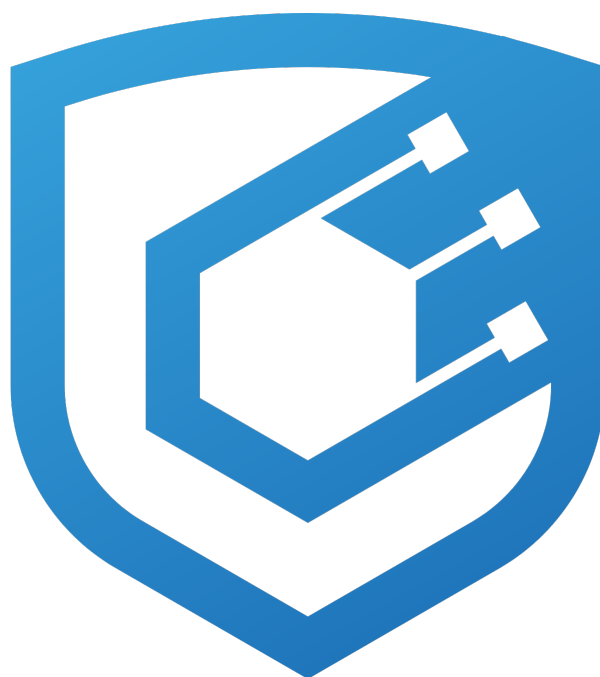




Matrixdock XAGM EVM Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Jorge](#)

[ChainDefenders](#) ([0x539](#), [PeterSR](#))

March 4, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	4
7.1	Low Risk	4
7.1.1	Off by one delay in mint execution	4
7.1.2	Under-collateralization due to reserve feed changes and cross-chain mint budget updates . .	4
7.1.3	Missing zero amount check can lead to a DoS of fee collection mechanism	6
7.2	Informational	8
7.2.1	Unused PrecisionLost error	8
7.2.2	Missing event emissions on governance revocation functions	8
7.2.3	Missing zero address validation during initialization	9
7.2.4	Blocked user can front-run Mtoken::addToBlockedList to transfer tokens	9
7.2.5	Fee collector can mint above annual rate in MtokenMain::reconcileSupply	9
7.3	Gas Optimization	11
7.3.1	Move the sender validation before state changing calls	11
7.3.2	Redundant modifier in Mtoken::multiTransfer	11

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Matrixdock is a Real World Asset (RWA) Tokenization protocol which has tokenized US Treasuries as STBT and physical gold as XAUm.

The purpose of this audit is to perform a security review of a new physical silver tokenization as XAGm, based on the codebase of previously-audited XAUm with some changes and enhancements.

5 Audit Scope

The audit scope was limited to:

```
xagm-evm/contracts/MToken.sol
xagm-evm/contracts/MTokenMain.sol
xagm-evm/contracts/MTokenSide.sol
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the [XAGM EVM](#) smart contracts provided by [Matrixdock](#). In this period, a total of 10 issues were found.

The findings consist of 3 Low severity issues with the remainder being informational and gas optimizations.

Centralization Risks

The `MToken::forcedTransfer` function allows the contract owner to unilaterally transfer tokens from any account to any account without restriction or validation. The function uses the `onlyOwner` modifier to gate access, but performs an arbitrary `_transfer` operation that moves tokens from `_from` to `_to` regardless of the sender's account

status or any other condition. This creates an unrestricted mechanism through which the owner can redistribute tokens at will, independent of the actual token holders' consent or the protocol's normal operational logic.

Summary

Project Name	XAGM EVM
Repository	RWA-Contracts
Commit	f905bb00d668...
Fix Commit	884ea2dc960a...
Audit Timeline	Feb 24th - Feb 26th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	3
Informational	5
Gas Optimizations	2
Total Issues	10

Summary of Findings

[L-1] Off by one delay in mint execution	Acknowledged
[L-2] Under-collateralization due to reserve feed changes and cross-chain mint budget updates	Acknowledged
[L-3] Missing zero amount check can lead to a DoS of fee collection mechanism	Resolved
[I-1] Unused <code>PrecisionLost</code> error	Resolved
[I-2] Missing event emissions on governance revocation functions	Acknowledged
[I-3] Missing zero address validation during initialization	Acknowledged
[I-4] Blocked user can front-run <code>Mtoken::addToBlockedList</code> to transfer tokens	Acknowledged
[I-5] Fee collector can mint above annual rate in <code>Mtoken-Main::reconcileSupply</code>	Acknowledged
[G-1] Move the sender validation before state changing calls	Acknowledged
[G-2] Redundant modifier in <code>Mtoken::multiTransfer</code>	Resolved

7 Findings

7.1 Low Risk

7.1.1 Off by one delay in mint execution

Description: The `MToken::mintTo` function implements a delay mechanism to prevent immediate token minting. When a mint request is first submitted, the effective time is set to `block.timestamp + delay`. However, when checking if enough time has passed, the function uses a strict less-than comparison: `if (et < block.timestamp)`. This creates an off-by-one timing issue where a mint request cannot be executed at the exact block when the delay expires. If the effective time `et` equals `block.timestamp` (meaning the delay period has technically elapsed), the condition evaluates to false, causing the function to revert with `TooEarlyToExecute`. The mint can only succeed in the block after the effective time, requiring an additional block to pass beyond the intended delay period.

Impact: This timing discrepancy delays the execution of approved mint requests by one additional block. While this doesn't constitute a security vulnerability or cause loss of funds, it creates operational friction and unpredictability. Users and operators expecting a mint to execute at a specific time will encounter an unexpected revert and need to retry in the next block. This could disrupt time-sensitive operations, particularly in scenarios involving cross-chain coordination or time-dependent liquidity management. The extra block delay could also expose the protocol to front-running or other timing-dependent attacks if not properly accounted for in higher-level orchestration logic.

Recommended Mitigation: Change the comparison operator from strict less-than (`<`) to less-than-or-equal (`<=`) to allow execution at the exact block when the delay period expires:

```
if (et <= block.timestamp) {
    delete requestMap[req]; // clear the record
} else {
    revert TooEarlyToExecute(receiver, amount, nonce);
}
```

Matrixdock: Acknowledged; the off-by-one mint request block is acceptable.

7.1.2 Under-collateralization due to reserve feed changes and cross-chain mint budget updates

Description: The Proof-of-Reserve (PoR) invariant in `MTokenMain` is enforced only at specific mint-budget increase points using the current reserve value returned by `getReserve`. However, the reserve is **mutable over time**, either via **oracle feed updates** (e.g., `setReserveFeed`) or custodian adjustments, and cross-chain mint budget increases can happen **without re-validation** on Ethereum.

This creates two main risk vectors:

1. **Reserve Feed Changes:** Switching to a new reserve feed can immediately reduce the reported reserve, potentially leaving previously issued obligations undercollateralized.
2. **Cross-Chain Mint Budget Reception:** Satellite contracts (e.g., silver/xAGM chains) can increase obligations via `ccReceiveMintBudget` without validating against the latest reserve on Ethereum.

The system does **not automatically enforce the PoR invariant** upon feed changes or cross-chain mint budget reception. Consequently, previously satisfied obligations may silently exceed available reserves.

Impact: `* totalTokenObligation * ozPerToken` can exceed the current reserve after a feed change or cross-chain budget reception.

- Silent undercollateralization can occur without any on-chain revert or alert.
- Backing guarantees of “fully backed by bullion” can be violated.
- The system can continue processing transfers, burns, and cross-chain operations while undercollateralized.

This can happen **without malicious behavior**, purely due to legitimate oracle updates or cross-chain synchronization timing.

Proof of Concept: Scenario A — Reserve Feed Change in `MTokenMain`

1. Initial state:

- Current reserve feed reports 1000 oz
- `totalTokenObligation = 900`
- `ozPerToken = 1`

Validation during `_increaseMintBudget` passes:

```
900 oz <= 1000 oz
```

2. Owner calls `setReserveFeed(newFeed)` to switch to a new oracle feed.

3. New feed reports lower reserve:

- `new reserve = 850 oz`

4. Contract state remains:

- `totalTokenObligation * ozPerToken = 900 oz`
- Reported reserve = 850 oz

5. No on-chain check occurs during feed change → PoR invariant is violated silently.

Scenario B — Cross-Chain Mint Budget Reception (`ccReceiveMintBudget`)

1. Reserve feed reports 1000 oz

2. `totalTokenObligation = 900` (already undercollateralized if feed changed)

3. Cross-chain message calls `ccReceiveMintBudget` for +150 mint budget:

- `totalTokenObligation = 1050`

4. No PoR validation occurs at message receipt.

5. Final state:

```
Required backing: 1050 oz  
Reported reserve: 1000 oz
```

System is now further undercollateralized.

Recommended Mitigation: Centralize PoR Invariant Check

Add a reusable invariant check in `MTokenMain`:

```
function _assertFullyBacked() internal view {  
    uint256 required = (totalTokenObligation * ozPerToken()) / OZ_RATIO_BASE;  
    int256 reserve = getReserve();  
    require(  
        reserve > 0 && int256(required) <= reserve,  
        "Reserve insufficient"  
    );  
}
```

Enforce PoR on All Obligation Increases

Apply `_assertFullyBacked` to every path that **increases obligations**:

In `_increaseMintBudget`:

```
function _increaseMintBudget(uint112 mintBudgetDelta) private {  
    uint112 newTokenObligation = totalTokenObligation + mintBudgetDelta;  
    totalTokenObligation = newTokenObligation;  
    _assertFullyBacked();  
    mintBudget += mintBudgetDelta;  
}
```

In reconcileSupply:

```
_increaseMintBudget(amount);  
_assertFullyBacked();
```

In ccReceiveMintBudget (satellite contract):

```
function ccReceiveMintBudget(uint112 amount) internal {  
    totalTokenObligation += amount;  
    _assertFullyBacked(); // validate against latest reserve  
    mintBudget += amount;  
}
```

Validate PoR Upon Reserve Feed Change

Enforce invariant whenever setReserveFeed is called:

```
function setReserveFeed(address addr) public onlyOwner {  
    _checkZeroAddress(addr);  
    uint64 et = etNextReserveFeed;  
    if (addr == nextReserveFeed && et != 0 && et < block.timestamp) {  
        reserveFeed = addr;  
        _assertFullyBacked(); // validate immediately  
        emit SetReserveFeedEffectuated(addr);  
    } else {  
        nextReserveFeed = addr;  
        etNextReserveFeed = uint64(block.timestamp) + delay;  
        emit SetReserveFeedRequest(reserveFeed, addr, etNextReserveFeed);  
    }  
}
```

Matrixdock: Acknowledged. For Reserve feed, it is kept as mutable for future third-party PoR provider change optionality. For cross-chain mint budget reception, satellite contracts on other chains are kept without re-validation of ETH PoR for possibilities of non-interoperable chains. Our backend keeps track of total supply across all chains, and enforces off-chain safeguards.

7.1.3 Missing zero amount check can lead to a DoS of fee collection mechanism

Description: The MTokenMain::reconcileSupply function lacks validation to prevent execution with a zero amount parameter. Currently, the function accepts any amount value, including zero, and proceeds to execute all logic steps including the critical state update of lastReconcileTime. If reconcileSupply(0) is called, the function will:

1. Pass the _increaseMintBudget(0) call without issue (trivial operation)
2. Execute mintBudget = mintBudget - 0 (no-op)
3. Call _mint(feeCollector, 0), which may emit a spurious Transfer event
4. Most critically: update lastReconcileTime to the current day's boundary

Since the reconcileSupply function includes a check if (_lastReconcileTime == newReconcileTime) to prevent multiple reconciliations within the same day, invoking the function with zero amount consumes the day's single allowed reconciliation slot without actually minting any fee tokens.

Impact: This vulnerability enables a denial-of-service attack where the fee collector can prevent by mistake legitimate fee collection for an entire 24-hour period by calling reconcileSupply(0). This disrupts the protocol's fee accrual mechanism, as actual fee minting will be blocked until the next day.

Recommended Mitigation: Add a zero-value check at the beginning of the reconcileSupply function to revert if amount == 0:

```
function reconcileSupply(uint112 amount) public onlyFeeCollector {  
    if (amount == 0) {
```

```

        revert ZeroValue();
    }

    // check and update lastReconcileTime
    uint64 _lastReconcileTime = lastReconcileTime;
    uint64 newReconcileTime = currentDayStartTime();
    if (_lastReconcileTime == newReconcileTime) {
        revert TooEarlyToReconcile();
    }
    lastReconcileTime = newReconcileTime;

    _increaseMintBudget(amount);
    mintBudget = mintBudget - amount;
    _mint(feeCollector, amount);
    emit ReconcileSupply(_lastReconcileTime, newReconcileTime, amount);
}

```

This ensures that `lastReconcileTime` is only updated when a meaningful fee reconciliation actually occurs, preventing the griefing attack vector and maintaining the integrity of the daily reconciliation schedule.

Matrixdock: Fixed in commit [6d499a](#).

Cyfrin: Verified.

7.2 Informational

7.2.1 Unused PrecisionLost error

Description: The PrecisionLost error in the MToken contract is defined in the contract's error definitions but is never used or thrown anywhere in the codebase. This is a code hygiene issue that creates dead code. The error definition suggests an intention to handle precision loss scenarios, particularly in mathematical operations involving division. However, the contract performs several arithmetic operations with potential precision loss - such as in `getOzAmount` and `ozPerToken` - without ever invoking this error. This discrepancy indicates either incomplete implementation (where precision checks were planned but not executed) or unnecessary code that should be cleaned up.

Recommended Mitigation: Two approaches are recommended:

1. **If precision loss detection is intended:** Identify and implement logic to detect precision loss in mathematical operations. Common scenarios include:

- When `(tokenAmount * ozPerToken()) % OZ_RATIO_BASE != 0` in `getOzAmount()`
- When `(annualFeeRate * daysElapsed) % DAYS_PER_YEAR != 0` in `ozPerToken()`

Add checks before these operations and throw `PrecisionLost()` if significant precision loss would occur. Alternatively, use `SafeMath` libraries or implement rounding strategies to minimize precision loss.

2. **If precision loss detection is not needed:** Remove the unused error definition entirely:

```
// Remove this line:  
error PrecisionLost();
```

This keeps the codebase clean and reduces confusion for future maintainers. Document in comments why precision loss is acceptable in these specific calculations if that decision is intentional.

Choose the approach based on whether precision loss in fee calculations or oz-to-token conversions represents a risk that requires protection in your protocol.

Matrixdock: Fixed in commit [81b76e](#).

Cyfrin: Verified.

7.2.2 Missing event emissions on governance revocation functions

Description: The following functions in `MTokenMain` modify critical the state but **do not emit any events**:

```
function revokeNextReserveFeed() public onlyRevoker {  
    etNextReserveFeed = 0;  
}  
  
function revokeNextFallbackFeed() public onlyRevoker {  
    etNextFallbackFeed = 0;  
}  
  
function revokeFeeCollector() public onlyRevoker {  
    etNextFeeCollector = 0;  
}  
  
function increaseMintBudget(uint112 mintBudgetDelta) public onlyOperator {  
    _increaseMintBudget(mintBudgetDelta);  
}  
  
function decreaseMintBudget(uint112 mintBudgetDelta) public onlyOperator {  
    uint112 newTokenObligation = totalTokenObligation - mintBudgetDelta;  
    mintBudget -= mintBudgetDelta;  
    totalTokenObligation = newTokenObligation;  
}
```

Because no events are emitted, off-chain systems, indexers, and governance monitors cannot detect that these pending changes have been revoked. This breaks consistency with the corresponding `set*` functions, which do emit events to announce scheduled changes.

For contracts relying on delayed governance actions, revocations are state-changing operations and should be observable on-chain.

Recommendation:

Emit explicit revocation events for each action.

Matrixdock: Acknowledged; we will add these events in future versions.

7.2.3 Missing zero address validation during initialization

Description: The `MTokenMain::__MTOKENMAIN_init`, `MToken::__MTOKEN_init` and `MToken::__MTOKEN_init_unchained` functions accepts several critical address parameters (`_owner`, `_operator`, `_reserveFeed`, and `_feeCollector`) and assign them directly to state variables without validating that they are not the zero address. While both the contracts implements a `_checkZeroAddress` validation function and consistently use it in other setter functions (such as `MTokenMain::setReserveFeed`, `MTokenMain::setFallbackFeed`, `MTokenMain::setFeeCollector`, `MToken::setMessenger`, `MToken::setRevoker`, `MToken::setOperator`), this validation is notably absent during initialization. If any of these critical addresses are mistakenly set to the zero address during contract initialization, it would result in a permanently misconfigured contract since these parameters control fundamental protocol operations like ownership, minting, reserve validation, and fee collection.

Recommended Mitigation: Add zero address validation checks for all critical address parameters in the given functions before they are assigned to state variables.

Matrixdock: Acknowledged; we will carefully verify it when deploying the contract.

7.2.4 Blocked user can front-run `Mtoken::addToBlockedList` to transfer tokens

Description: Blocking is applied in `Mtoken::addToBlockedList` by setting `isBlocked[_user] = true`. All transfers that move the user's tokens are gated by `Mtoken::onlyNotBlocked` and/or `Mtoken::_checkBlocked`, which read `isBlocked`. a user who sees that transaction in the mempool can front-run it with a transfer of their tokens to another address. When their transfer runs first, `isBlocked[user]` is still false, so the transfer succeeds; once `Mtoken::addToBlockedList` is applied, the user is blocked but their balance is already zero and the tokens are in the other address (e.g. another wallet they control).

```
function addToBlockedList(address _user) public onlyOperator {
    isBlocked[_user] = true;
    emit BlockPlaced(_user);
}
```

Recommended Mitigation: Use flashbots (private mempool) when blocking users.

Matrixdock: Acknowledged; we will pay attention to this issue during operation.

7.2.5 Fee collector can mint above annual rate in `MtokenMain::reconcileSupply`

Description: In `MTokenMain::reconcileSupply` lets the fee collector mint fee tokens once per day. The only on chain constraint on amount is the reserve check inside `MTokenMain::_increaseMintBudget` (total token obligation must not exceed the proof of reserve). There is no check that amount is consistent with the configured annual fee rate.

`annualFeeRate` is used only in `MToken.sol` to decrease `ozPerToken` over time (fee accrual via devaluation). The comment in `MToken.sol` states that the daily fee rate is `annualFeeRate / DAYS_PER_YEAR`, but that relationship is never used to cap the mint in `reconcileSupply`

Recommended Mitigation: Derive a maximum allowed daily fee from the annual rate and enforce it in `reconcileSupply`.

Matrixdock: Acknowledged; this will be checked on our backend calculations, we have a complete reconcile-Supply quantity verification against the fee rate, totalSupply and number of days elapsed.

7.3 Gas Optimization

7.3.1 Move the sender validation before state changing calls

Description: In the `MToken::ccSendToken` function, the code performs an expensive token burn operation (`_burn(sender, value)`) before checking whether the sender is a blocked account. The blocked status check occurs later in the `msgOfCcSendToken` function, which is called after the burn has already taken effect. If a blocked sender calls this function, the transaction will revert, but the gas consumed by the `_burn` operation will not be refunded. This results in unnecessary gas expenditure for failed transactions from blocked accounts.

Recommended Mitigation: Move the `_checkBlocked(sender)` call to the beginning of the `ccSendToken()` function, before any state-modifying operations like `_burn`. This ensures that if the sender is blocked, the function will revert early without wasting gas on token burning. The check should be placed at least before `_burn`, allowing the transaction to fail as cheaply as possible for blocked accounts.

Matrixdock: Acknowledged; we will keep it as is for now, likelihood of blocked accounts is low and consumed gas is acceptable.

7.3.2 Redundant modifier in `MToken::multiTransfer`

Description: `MToken::multiTransfer` is protected by `onlyNotBlocked`, so the caller is already checked. Each iteration then calls `MToken::transfer`, and `transfer` also uses `onlyNotBlocked`, so the same sender is checked again on every loop iteration. The block check is redundant in `MToken::multiTransfer`.

```
function multiTransfer(
    address[] calldata _recipients,
    uint256[] calldata _values
) public onlyNotBlocked {//@audit the transfer already check for blocked sender
    if (_recipients.length != _values.length) {
        revert ArgsMismatch();
    }
    for (uint256 i = 0; i < _recipients.length; i++) {
        transfer(_recipients[i], _values[i]);
    }
}
```

Recommended Mitigation: Remove the `onlyNotBlocked` modifier from `MToken::multiTransfer` and rely on the check inside `transfer`

Matrixdock: Fixed in commit [cd5cd2](#).

Cyfrin: Verified.